Chapitre 1 — Variables et types de base

En Python, une variable est un nom qui désigne une valeur. On ne déclare pas son type : Python le déduit de la valeur affectée. Les types de base sont les entiers, les nombres à virgule flottante, les chaînes de caractères et les booléens. La fonction type permet de connaître le type d'une valeur à tout moment.

Les chaînes de caractères sont immuables : on ne peut pas modifier une lettre en place, on crée une nouvelle chaîne. Les f-strings permettent d'insérer des valeurs dans un texte de façon lisible, en préfixant la chaîne par la lettre f et en plaçant les expressions entre accolades.

Les annotations de type, comme nombre: int, ne changent pas l'exécution du programme. Elles servent à documenter le code et permettent à des outils de détecter des erreurs avant de lancer le programme.

Chapitre 2 — Structures de contrôle

Les conditions s'écrivent avec if, elif et else. Le bloc de code concerné est défini par l'indentation, et non par des accolades comme dans d'autres langages. Une indentation incohérente provoque donc une erreur.

La boucle for parcourt les éléments d'une séquence : une liste, une chaîne, ou un intervalle produit par range. La boucle while répète un bloc tant qu'une condition est vraie ; il faut s'assurer que cette condition finit par devenir fausse, sinon la boucle ne s'arrête jamais. Les mots-clés break et continue permettent respectivement de sortir de la boucle ou de passer directement à l'itération suivante.

La fonction enumerate donne à la fois l'indice et l'élément, et zip parcourt plusieurs séquences en parallèle. Ces deux fonctions évitent de manipuler des indices à la main, ce qui est une source fréquente d'erreurs.

Chapitre 3 — Fonctions

Une fonction regroupe des instructions sous un nom pour pouvoir les réutiliser. Elle se définit avec le mot-clé def, reçoit des paramètres et renvoie une valeur avec return. Une fonction sans return renvoie None.

Une bonne fonction fait une seule chose et porte un nom qui décrit cette chose. Elle est courte, ce qui la rend facile à lire et à tester. Sa docstring, placée juste sous la ligne def, explique en une phrase ce qu'elle fait.

Les paramètres peuvent avoir une valeur par défaut. Attention : une valeur par défaut mutable, comme une liste vide, est créée une seule fois et partagée entre tous les appels, ce qui provoque des bugs surprenants. On utilise None comme valeur par défaut à la place, puis on crée la liste dans le corps de la fonction.

Une fonction pure ne dépend que de ses paramètres et ne modifie rien à l'extérieur. Elle donne toujours le même résultat pour les mêmes entrées, ce qui la rend très simple à tester.

Chapitre 4 — Listes et dictionnaires

La liste est une séquence ordonnée et modifiable. On ajoute un élément avec append, on accède à un élément par son indice, et on extrait une partie avec le découpage. Le dictionnaire associe des clés à des valeurs ; l'accès par clé est très rapide, même avec beaucoup d'éléments.

Les compréhensions de liste permettent de construire une liste à partir d'une autre en une ligne. Elles sont pratiques, mais une compréhension trop longue devient illisible : dans ce cas, une boucle classique est préférable.

Chapitre 5 — Tests avec pytest

Un test automatique vérifie qu'une fonction renvoie le résultat attendu. Avec pytest, un test est une simple fonction dont le nom commence par test, et la vérification s'écrit avec le mot-clé assert. Pytest découvre et lance tous les tests du projet avec une seule commande.

Les fixtures préparent ce dont les tests ont besoin, par exemple une base de données temporaire, et la nettoient ensuite. Elles évitent de répéter le même code de préparation dans chaque test et garantissent que les tests ne s'influencent pas entre eux.

On écrit un test pour chaque règle métier et pour chaque cas limite : une liste vide, une valeur nulle, une date dans le passé. Un test qui échoue avant la correction puis réussit après est la meilleure preuve qu'un bug est réglé.